NCCLbpf: Verified, Composable Policy Execution for GPU Collective Communication

Anonymous Author(s)

ABSTRACT

NCCL is the de facto standard library for GPU collective communication in large-scale distributed training. Its growing plugin ecosystem allows operators to customize collective behavior, but plugins execute as unverified native code that can crash training jobs, share no state across extension points, and cannot be updated without restarting. We observe that NCCL's plugin architecture is structurally analogous to kernel extension points where eBPF already provides verified, high-performance extensibility. We present NCCLbpf, which embeds a userspace eBPF runtime inside NCCL's tuner, profiler, and net plugin interfaces to provide load-time verification, typed cross-plugin maps, and atomic policy hot-reload, all without modifying NCCL. On NCCL 2.29.7, NCCLbpf adds 52–103 ns per tuner decision (<0.06% of collective latency) and negligible overhead on the net data path, rejects all tested unsafe programs at load time, and eliminates intermittent 2× AllGather throughput degradation on our testbed.

1 INTRODUCTION

Large-scale GPU training depends on collective communication performance. Frameworks such as Megatron-LM [15], DeepSpeed [12], and PyTorch FSDP [19] rely on NCCL [9] for all inter-GPU data movement. The choice of algorithm (ring vs. tree), protocol (LL vs. Simple), and channel count significantly affects collective latency [2, 3]. NCCL exposes a plugin system (tuner, profiler, net, and env plugins) that allows operators to customize collective behavior, and major cloud providers already ship custom tuner plugins [2].

However, every plugin executes as unverified native code in NCCL's address space: a null-pointer dereference crashes the training job, an infinite loop hangs the communicator, and an off-by-one error can silently corrupt internal state. The tuner and profiler are independent extension points with no shared state mechanism, precluding closed-loop control. Updating a plugin requires restarting the job, leaving operators no way to iterate on policies at runtime.

We observe that NCCL's plugin architecture is structurally analogous to kernel extension points (packet filters, storage hooks, memory-management policies [17, 21, 22]) where eBPF already provides verified, high-performance extensibility. Both demand safe extensibility of a performance-critical runtime: NCCL's net plugin directly parallels eBPF's original packet-processing domain, while the tuner and profiler plugins are policy decision points analogous to kernel tracing hooks. eBPF's verify-then-execute model addresses all three gaps above: static verification catches crash-inducing bugs *before* execution (not merely sandboxing them at runtime), typed maps provide structured cross-plugin state, and atomic program replacement enables live policy updates, all while preserving near-native JIT performance.

NCCLbpf targets all three plugin types: tuner and profiler for policy decisions (52–103 ns overhead), and net for data-plane hooks with negligible throughput impact. NCCLbpf embeds bpftime [18], a userspace eBPF runtime, inside a standard NCCL plugin. Policy authors write restricted C compiled to BPF bytecode, verified at load time, and JIT-compiled to native code, all with zero modifications to NCCL. This paper makes four contributions:

(1) **NCCLbpf**, a system that brings verified eBPF policy execution to NCCL's plugin interface (§3–4).
(2) A map-based composability model that enables closed-loop profiler-to-tuner adaptation, a capability absent from NCCL's native plugin architecture (§3, §5.3).
(3) Atomic policy hot-reload with 0.774 µs swap time and zero call loss across 400,000 invocations (§3, §5.2).
(4) An evaluation on NCCL 2.29.7 showing 52–103 ns CPU overhead per decision (<0.06% of collective latency), end-to-end GPU overhead within measurement noise, and elimination of intermittent 2× AllGather throughput degradation (§5).

2 BACKGROUND AND MOTIVATION

NCCL and its plugin system. NCCL [9] implements GPU collective primitives (AllReduce, AllGather, Broadcast, etc.) optimized for multi-GPU interconnects. For each collective call, the *tuner* plugin's `getCollInfo` receives the collective type, message size, and rank topology, and selects an algorithm (ring, tree), protocol (LL, LL128, Simple [6]), and channel count by modifying a cost table. The *profiler* plugin receives timestamped event callbacks; the *net* plugin controls network transport; and the *env* plugin sets runtime parameters. All four are loaded via `dlopen` as shared libraries. The tuner API has evolved rapidly, from v2 (simple integer outputs) to v5 (in-place 2D float cost table modification) in two years, and major cloud providers ship custom tuner plugins [2]. Despite this growing ecosystem, plugins remain independent extension points with no cross-plugin communication and no safety verification.

The safety gap. Native plugins execute with full process privileges. NCCL's own inspector plugin has caused production training crashes due to use-after-free and deadlock bugs [10], and profiler plugins have triggered segfaults on multi-GPU nodes [11]. At scale, such failures are costly: Llama 3 pre-training on 16,384 GPUs experienced 466 job interruptions in 54 days [4], and analyses of production GPU clusters find that ~30% of training jobs fail, with programming and configuration errors as the dominant cause [7]. The absence of safe hot-reload further compounds the risk: rolling out plugin updates across a fleet requires restarting every training job.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

eBPF as safe extension mechanism. Extended BPF (eBPF) [16] is an execution framework, originally implemented in the Linux kernel for packet filtering and tracing, that statically verifies programs for memory safety and bounded execution before JIT-compiling them to native code. eBPF has been adopted as a safe-extension mechanism across storage [21], networking [8, 22], memory management [17], and GPU driver-level resource management [20]. Recent userspace runtimes [18] extend its reach beyond the kernel, enabling eBPF execution inside application processes without kernel privileges. Three properties make eBPF well-suited to NCCL policy execution: (1) load-time verification catches memory-safety and termination bugs that cause the crashes described above; (2) typed maps provide concurrent state sharing between profiler and tuner without ad hoc shared memory; and (3) atomic program replacement enables verified hot-reload.

3 DESIGN

3.1 Design Tensions

T1: Safety vs. overhead. NCCLbpf verifies only at load time (1–5 ms, amortized over the job lifetime), relying on the invariant that verified BPF bytecode, once JIT-compiled, cannot violate its safety guarantees at runtime. The tradeoff: verification catches memory safety and termination bugs, not semantic correctness.

T2: Structured sharing vs. flexibility. All cross-component state sharing uses typed eBPF maps with atomic access semantics. Maps constrain policies to fixed-size keys/values, but NCCL policy state is inherently simple (per-communicator scalars), and structured sharing eliminates the locking bugs that plague ad hoc native plugin state sharing.

T3: Availability vs. consistency during updates. Hot-reload atomically swaps the function pointer; any in-progress call completes under the old policy, the next call uses the new one. No call is lost. If the new policy fails verification, the old policy continues. The tradeoff: different threads may briefly execute different policies, which is acceptable because collective decisions are independent across calls.

T4: Compatibility vs. capability. NCCLbpf operates within NCCL's existing plugin ABI, requiring no source modifications, no custom builds, and no kernel modules. The action space is limited to algorithm, protocol, and channel count selection. Deeper interventions are outside scope, but the exposed knobs cover the most impactful tuning decisions. NCCLbpf selects among NCCL's built-in algorithms; it does not implement custom collective algorithms.

3.2 Threat Model

Policies are written by platform operators, not untrusted tenants. The eBPF verifier (PREVAIL-based) checks memory safety, bounded execution, stack safety, and helper whitelisting. The system does *not* protect against resource exhaustion, side channels, bugs in the trusted computing base (bpftime JIT, PREVAIL verifier, host plugin), or semantically incorrect decisions. This provides defense in depth (eliminating crashes, hangs, and memory corruption), analogous to the Linux kernel's eBPF model.

3.3 Architecture

Figure 1 shows the architecture. Policy authors write restricted C compiled to BPF ELF objects. At load time, each program is verified and JIT-compiled to x86-64 code. The map subsystem provides intra-policy state and cross-plugin communication: a profiler eBPF program writes latency observations; the tuner reads them. Hot-reload verifies and JIT-compiles a replacement, then atomically swaps the function pointer via compare-and-swap.

Policy programming model. A policy takes a `policy_context` (collective type, message size, rank count) and writes algorithm, protocol, and channel count decisions. The verifier ensures policies only read input fields and write output fields. Map lookups require null checks before dereference; omitting one causes load-time rejection.

Listing 1: Profiler-to-tuner closed loop. The profiler (top) writes latency into a shared eBPF map; the tuner (bottom) reads it for adaptive channel selection. Null checks (lines 5, 14) are enforced by the verifier.

```
1  /* --- profiler eBPF program --- */
2  SEC("profiler")
3  int record_latency(struct profiler_context *ctx) {
4    __u32 key = ctx->comm_id;
5    struct latency_state *st =
6      bpf_map_lookup_elem(&latency_map, &key);
7    if (!st) return 0;
8    st->avg_latency_ns = ctx->latency_ns;
9    st->channels = ctx->n_channels;
10   return 0;
11 }
12 /* --- tuner eBPF program --- */
13 SEC("tuner")
14 int size_aware_adaptive(struct policy_context *ctx) {
15   __u32 key = ctx->comm_id;
16   struct latency_state *st =
17     bpf_map_lookup_elem(&latency_map, &key);
18   if (!st) { ctx->n_channels = 4; return 0; }
19   if (ctx->msg_size <= 32 * 1024)
20     ctx->algorithm = NCCL_ALGO_TREE;
21   else
22     ctx->algorithm = NCCL_ALGO_RING;
23   ctx->protocol = NCCL_PROTO_SIMPLE;
24   if (st->avg_latency_ns > 1000000)
25     ctx->n_channels = min(st->channels + 1, 16);
26   else
27     ctx->n_channels = st->channels;
28   return 0;
29 }
```

The two programs share `latency_map` via the eBPF map subsystem, a capability absent from NCCL's native plugin architecture, where tuner and profiler have no shared state. Null checks on map lookups (lines 5 and 14) are enforced by the verifier; omitting either causes load-time rejection.

4 IMPLEMENTATION

NCCLbpf is implemented in approximately 2,500 lines of C/C++ (plugin host) and 500 lines of restricted C (policy programs).

bpftime integration. We embed bpftime [18] as the userspace eBPF runtime. bpftime provides an LLVM-based JIT compiler, a PREVAIL-based [5] static verifier, and a map subsystem. The verifier runs in the same address space as the plugin, adding approximately 1–5 ms of one-time startup overhead amortized over the lifetime of the training job. The LLVM JIT produces optimized x86-64 code, narrowing the gap to native performance.

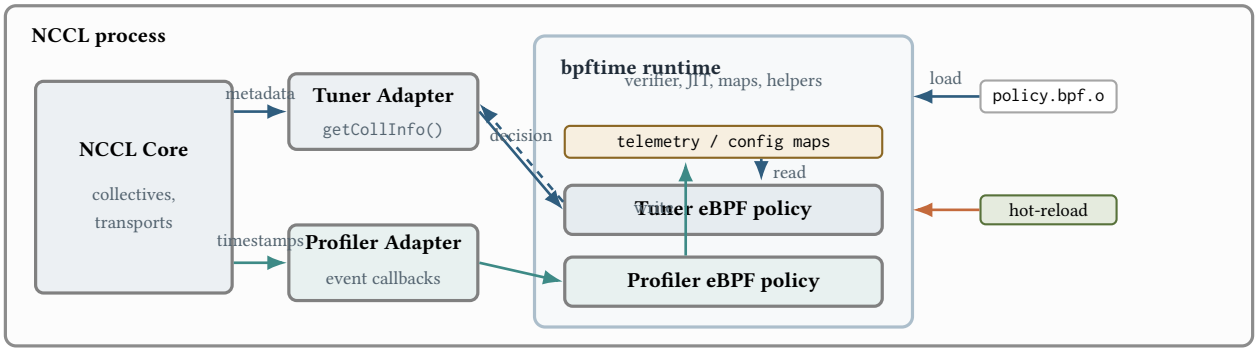

Figure 1: NCCLbpf architecture. Policy programs are verified via PREVAIL, JIT-compiled by bpftime, and execute inside the NCCL process. Profiler and tuner communicate through shared typed eBPF maps. The entire system loads as standard NCCL plugins.

Plugin registration. NCCLbpf registers itself as both a tuner and profiler plugin via NCCL's `ncclTunerPlugin_v3` (or v5) and `ncclProfilerPlugin_v1` (or v6) interfaces. The tuner plugin's `getCollInfo()` function constructs a `policy_context`, invokes the JIT-compiled eBPF function, and copies the output fields into NCCL's result structure. The profiler plugin's event callbacks extract latency information and update the shared eBPF maps.

NCCL integration challenges. NCCL's tuner API uses cost arrays rather than direct algorithm IDs: the tuner sets costs to zero for preferred choices and to a sentinel value (`1e9`) for others, allowing NCCL to fall back gracefully if the requested combination is unavailable. Our integration translates eBPF policy outputs into cost array entries. NCCL also passes a maximum channel count that the tuner must respect; our native baseline layer clamps the policy's request. Communicator identification required deriving a stable ID from the context pointer via hashing, since the communicator ID is not directly exposed in the tuner API as a simple integer.

Hot-reload mechanism. The active policy is stored as an atomic function pointer. Reload has three phases: (1) verify the new BPF program, (2) JIT-compile it, (3) atomic compare-and-swap on the pointer. The total reload cost is ~7.3 ms; only the final swap (0.774 µs) touches the hot path. The old pointer is retained until in-flight calls drain. If the replacement fails verification, the swap is aborted and the old policy continues; the system never enters an unverified state.

Native baseline for comparison. To provide a fair overhead comparison, we implemented a native C++ baseline that performs identical policy logic without the eBPF layer, compiled with the same optimization flags (`-O2`). The overhead difference between the native baseline and eBPF policies directly measures the cost of the eBPF dispatch and JIT execution layers, isolated from any policy logic cost.

5 EVALUATION

Testbed. CPU microbenchmarks run on a 24-core x86-64 platform with isolated cores; each benchmark performs 1 million calls and reports P50/P99 latencies. GPU experiments use a single NVIDIA RTX 5090 with CUDA 12.9 and NCCL 2.29.7, running 2 MPI ranks on the same host over socket transport.

Table 1: CPU microbenchmark (1M calls each). eBPF policies add 52–103 ns over native code. The overhead decomposes into eBPF dispatch (+51 ns), map lookup (+26 ns each), and map update (+25 ns each).

Policy	P50 (ns)	P99 (ns)	ΔP50
Native baseline	10	16	—
`noop`	61	74	+51
`size_aware_v2`	62	73	+52
`lookup_only`	88	104	+78
`lookup_update`	113	134	+103
`adaptive_channels`	112	136	+102
`slo_enforcer`	110	134	+100

5.1 Overhead

Table 1 reports per-call latency. The overhead is predictable: approximately $51 + 26n_{\text{lookup}} + 25n_{\text{update}}$ ns. Even the most complex policy (`slo_enforcer`) adds only 100 ns. For a 128 MB AllReduce (175 µs), this is 0.06% of collective latency.

End-to-end GPU measurements across 10 message sizes (8 B to 128 MB) confirm that the CPU-side overhead is invisible at the GPU level: all tested configurations (no plugin, `noop`, `size_aware_v2`, `slo_enforcer`) produce statistically indistinguishable AllReduce latencies.

5.2 Safety and Hot-Reload

We tested 14 eBPF programs against the PREVAIL-based verifier: 7 safe policies (including all policies in Table 1) were accepted; 7 unsafe programs, each targeting a distinct bug class (null-pointer dereference, out-of-bounds access, illegal helper, stack overflow, unbounded loop, input-field write, division by zero), were rejected at load time with actionable error messages.

The safety difference is concrete. We implemented the same null-dereference bug as both a native C plugin and an eBPF policy:

```
Native plugin: Signal: SIGSEGV (address 0x0)
                in getCollInfo() at native_bad_plugin.so
```

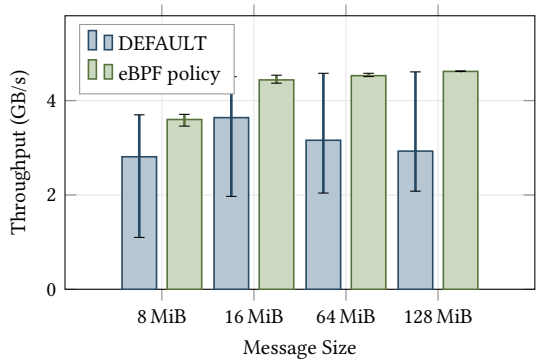

Figure 2: AllGather throughput stability (bars: mean of 3 runs; whiskers: min–max range). DEFAULT exhibits large run-to-run variance at all sizes; our eBPF policy eliminates it. RTX 5090, 2 ranks, socket transport.

```
eBPF policy:  VERIFIER REJECT: R0 is a pointer to
                map_value_or_null; must check != NULL
                before dereference at insn 7
```

The native plugin crashes the training job; the eBPF policy is caught before execution.

Hot-reload swaps the active policy atomically via compare-and-swap (0.774 µs swap time; ~7.3 ms including verification and JIT). Across 400,000 continuous invocations, we observe zero lost calls. If the replacement fails verification, the old policy continues uninterrupted.

5.3 Case Studies

Stability improvement. AllGather under NCCL's default configuration exhibits *high run-to-run variance*: across 3 independent runs at 128 MiB, 2 runs produced only 2.08–2.11 GB/s while the third achieved 4.61 GB/s (Figure 2). The degradation affects all message sizes, not just large transfers. The root cause is outside the scope of this paper; we treat the variance as a symptom that policy steering can address.

Our `size_aware_adaptive` policy (Listing 1) selects Simple protocol for all collectives, producing 4.62±0.01 GB/s at 128 MiB across all 3 runs. This demonstrates a policy value beyond throughput optimization: *improved stability* through consistent protocol selection. The policy adds zero measurable GPU-level overhead versus NCCL's default in its normal operating mode.

Profiler-to-tuner composability. To evaluate cross-plugin composability, we construct an adaptive channels policy that starts with a conservative channel count (nChannels = 2) and relies on profiler telemetry to adapt upward. Without the profiler, the tuner receives no samples and remains at 2 channels. With the profiler enabled, latency samples flow into a shared eBPF map, and the tuner ramps through 3 to 4 channels (the best count observed for this workload).

Over 5 runs of 128 MiB AllGather, the tuner-only configuration achieves 3.84 GB/s (pinned at 2 channels), while the combined configuration reaches 4.49±0.11 GB/s after adaptation, a 16.9% improvement. The adapted throughput matches NCCL's default (4.44±0.14 GB/s), showing that the feedback loop recovers optimal performance from a deliberately suboptimal cold start. This validates NCCLbpf's composability model: two independently deployed eBPF programs cooperate through shared typed maps, a capability absent from NCCL's native plugin architecture.

Net plugin extensibility. To validate that eBPF extends beyond the tuner/profiler control plane, we implement an eBPF-wrapped net plugin that interposes on NCCL's Socket transport. The wrapper forwards all transport operations to the built-in backend while executing a BPF program at each `isend`/`irecv` call to count bytes and connections via a shared eBPF map. Over 5 runs of 128 MiB AllReduce, the wrapped transport achieves 2.09±0.11 GB/s versus 2.13±0.03 GB/s for the unwrapped baseline (1.9% mean difference, within run-to-run noise). This confirms that eBPF hooks can operate on NCCL's data-plane path with negligible overhead, paralleling eBPF's established role in kernel packet processing.

6 RELATED WORK

Collective communication. MSCCL [3] and TACCL [14] synthesize static schedules offline; they do not address runtime policy selection or safety. AutoCCL [2] automates tuning via search but uses native code without verification. MSCCL++ [13] provides GPU-driven primitives at the mechanism layer. Hu et al. [6] provide a comprehensive analysis of NCCL's protocol variants and algorithms. gpu_ext [20] applies eBPF to GPU driver-level resource management; NCCLbpf targets the collective communication library layer, where policies govern algorithm and protocol selection rather than device resources. NCCLbpf is complementary: a policy can activate synthesized schedules or tuned configurations based on runtime conditions.

Alternative extension mechanisms. WebAssembly provides sandboxed execution but lacks eBPF's static termination and memory-safety guarantees; its overhead is typically higher due to bounds checks that eBPF eliminates statically. Out-of-process services provide strong isolation but add microseconds of IPC latency, an order of magnitude more than NCCLbpf's 52–103 ns overhead. Declarative DSLs cannot express stateful policies or feedback loops. Hardware isolation (Intel MPK) provides memory separation without verification. eBPF uniquely combines static verification with near-native JIT performance in a single model.

7 DISCUSSION

NCCLbpf currently covers tuner, profiler, and net plugins; extending coverage to the env plugin is straightforward. Our net plugin prototype wraps the built-in Socket backend; deeper integration (e.g., RDMA-aware policies) is future work. Our evaluation uses 2 ranks on a single node over socket transport; multi-node experiments with InfiniBand and large rank counts are needed to validate the approach at production scale. A higher-level policy DSL compiled to BPF would lower the barrier for ML engineers who are not familiar with eBPF programming.

The eBPF-for-extensions pattern may generalize beyond NCCL. AMD's RCCL [1] exposes a similar plugin architecture, suggesting that a cross-vendor port may be feasible. Other libraries with native-code extension points (MPI implementations, RDMA middleware)

face similar safety gaps and could benefit from the same verify-then-execute approach.

8 CONCLUSION

We presented NCCLbpf, a system that transforms NCCL's unsafe native plugin extension points into verified, composable policy hooks by embedding a userspace eBPF runtime. NCCL's plugin architecture is structurally analogous to kernel extension points where eBPF is already proven; NCCLbpf brings this model to GPU collective communication, covering tuner, profiler, and net plugins. NCCLbpf adds 52–103 ns per policy decision and negligible overhead on the net data path, catches crash-inducing bugs at load time, supports atomic hot-reload without call loss, and eliminates intermittent AllGather throughput degradation on our testbed, with zero NCCL modifications.

REFERENCES

[1] AMD. 2024. RCCL: ROCm Communication Collectives Library. https://github.com/ROCm/rccl.
[2] Zhihao Cai et al. 2024. AutoCCL: Automated Collective Communication Tuning for GPU Clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC).*
[3] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCL: Programmable Communication Schedules for GPU Clusters. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*
[4] Abhimanyu Dubey et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
[5] Elazar Gershuni, Nadav Amit, et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI).*
[6] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, et al. 2025. Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms. *arXiv preprint arXiv:2507.04786* (2025).
[7] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference (ATC).*
[8] Bing Liu et al. 2024. eTran: Efficient and Extensible Transparent Network Functions. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*
[9] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.
[10] NVIDIA. 2026. Deadlock/crash due to UAF in inspector plugin. https://github.com/NVIDIA/nccl/issues/2000.
[11] NVIDIA. 2026. Inspector bug: segfault encountered during training. https://github.com/NVIDIA/nccl/issues/1992.
[12] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.*
[13] Mehrdad Salehi et al. 2025. MSCCL++: A GPU-driven Communication Stack for Scalable AI Training. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys).*
[14] Aashaka Shah et al. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*
[15] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
[16] Nelson Billing Vieira et al. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys* (2020).
[17] Xueyuan Wang et al. 2025. PageFlex: eBPF-Based Programmable Page Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS).*
[18] Tong Yu, Yusheng Zheng, et al. 2025. bpftime: Userspace eBPF Runtime for Uprobe, Syscall and Kernel-Function Tracing. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*
[19] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Les Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Benjamin Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. In *Proceedings of the VLDB Endowment.*
[20] Yusheng Zheng, Tong Yu, Yiwei Yang, et al. 2025. gpu_ext: Extensible OS Policies for GPUs via eBPF. *arXiv preprint arXiv:2512.12615* (2025).
[21] Yuhong Zhong, Haoyu Zhu, et al. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*
[22] Yu Zhou et al. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*